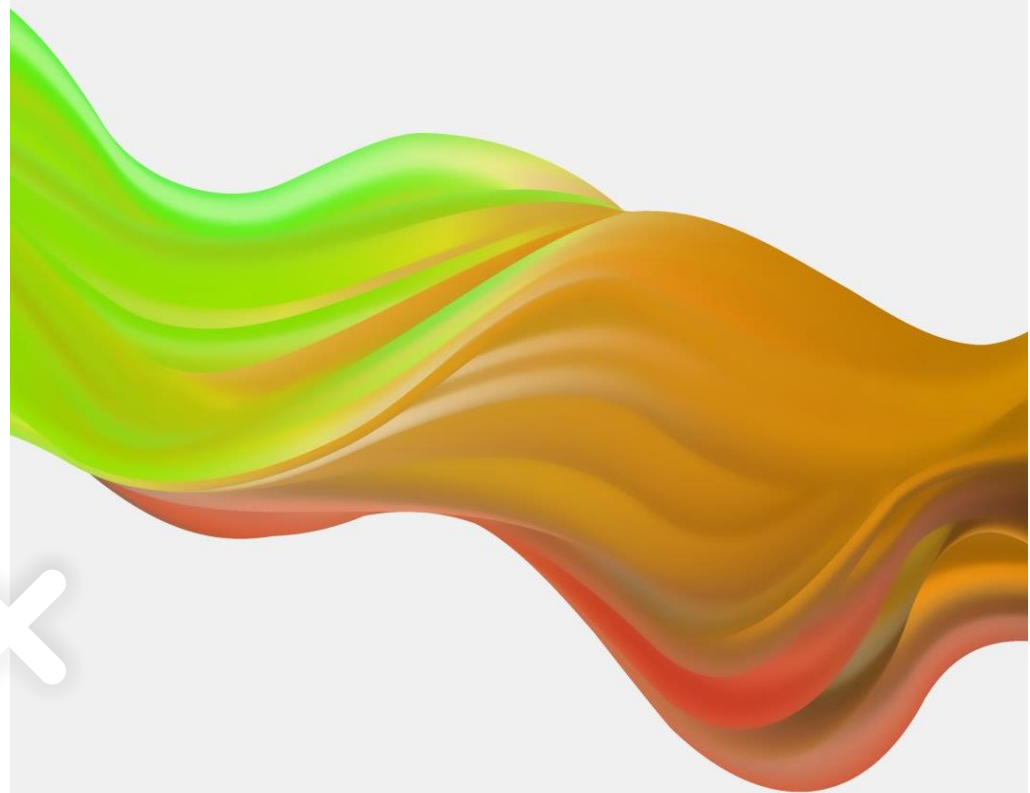




Computer Programming

MENG2020





REVIEW



Lesson Overview

In this lesson we will study:

- **Polynomials:** Functions of a special form that arise often in science.
- **Curve Fitting:** Finding the exact function of a specified form that represents the data with the least amount of error.
- **Interpolation:** Estimating values between data points.

Polynomials

- MATLAB represents a polynomial by a row vector of coefficients.
 - The first vector element is the coefficient of x to highest power (c^n)
 - The second vector element is c^{n-1}
 - The n th vector element is c^1
 - The $n+1^{th}$ vector element is c^0

- MATLAB represents a polynomial of degree n by a vector of length $n+1$.
- By degree we mean the highest power of the independent variable appearing in the polynomial.
- For example, in the polynomial $6x^2 + 8x - 7$ the degree of the polynomial is 2.

Polynomial Coefficients

- The vector must include all the polynomial coefficients, even those that are zero.

Examples:

Polynomial

$$8x + 5$$

$$2x^2 - 4x + 10$$

$$6x^2 - 150, \text{ MATLAB form: } 6x^2 + 0x - 150$$

$$5x^5 + 6x^2 - 7x, \text{ MATLAB form:}$$

$$5x^5 + 0x^4 + 0x^3 + 6x^2 - 7x + 0$$

MATLAB representation

$$p = [8 \ 5]$$

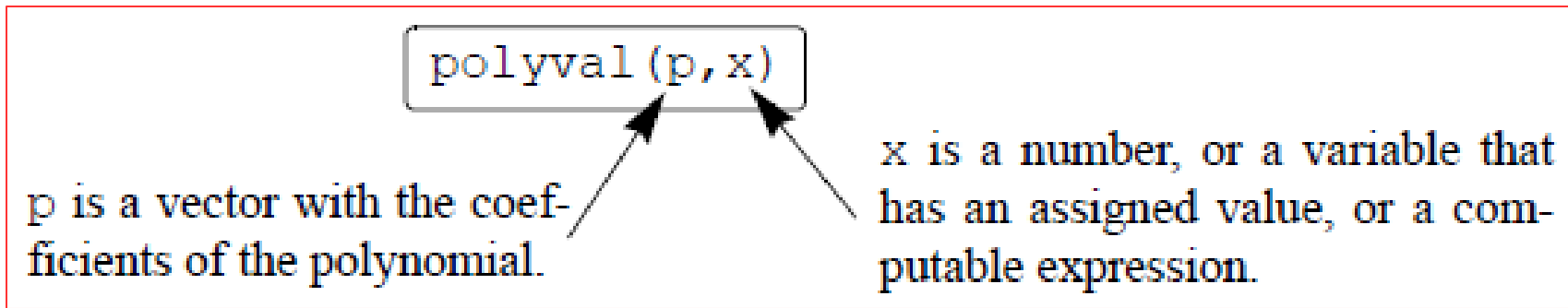
$$d = [2 \ -4 \ 10]$$

$$h = [6 \ 0 \ -150]$$

$$c = [5 \ 0 \ 0 \ 6 \ -7 \ 0]$$

Polynomials Values : The polyval Command

- The MATLAB command **polyval** computes the value of a polynomial at a point x (or at a series of points if x is a vector or a matrix).



- $\mathbf{p}$ is a vector of polynomial coefficients.
- $\mathbf{x}$ is a scalar, vector, or matrix of points at which the polynomial should be evaluated (done element-wise if x is a vector or a matrix).

Roots of a Polynomial

- The *roots* of a polynomial are the values of the independent variable that make the polynomial to be zero.
- A polynomial of degree n has exactly n roots, though some may be repeated.
- The MATLAB command **roots** finds the roots of a polynomial.

```
r = roots(p)
```

r is a column vector with the roots of the polynomial.

p is a row vector with the coefficients of the polynomial.

The roots Command : Example

- Consider a polynomial **p** expressed as a vector of coefficients.

p = [1 -3 -3 7 6]

- Get the roots of the polynomial into a vertical vector **rts** with the roots command.

rts = roots(**p**)

rts =

3.0000

2.0000

-1.0000

-1.0000

Roots of a Polynomial and the poly Command

- If, on the other hand, you know the roots of a polynomial but not the coefficients, you can get them with the **poly** command.
- Let's try that command with our vector of roots **rts**.

```
coeffs = poly(rts)
```

```
coeffs =
```

```
      1.0000    -3.0000    -3.0000  
7.0000      6.0000
```

- Not surprisingly, you can see that **coeffs** is the same vector as our original polynomial **p**.

Addition/Subtraction of Polynomials

- To add (subtract) two polynomials, you add (subtract) their vectors of coefficients.
- If one vector is shorter than the other, you must add enough zeros in front of the shorter vector so that it gets to the same size as the longer vector.

```
>> p1 = [5 -10 3];
```

```
>> p2 = [0 2 7];
```

```
>> p3 = p1 + p2
```

```
p3 = 5 -8 10
```

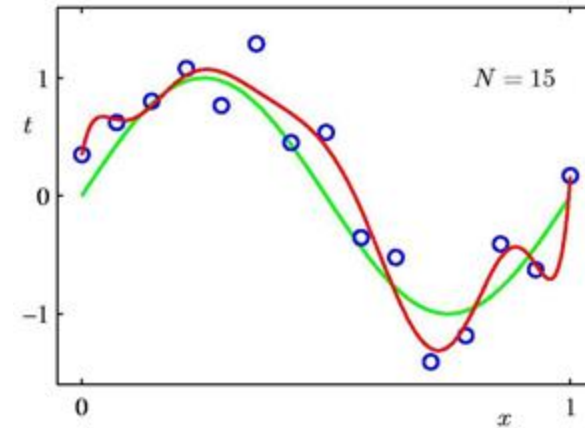
$$5x^2 - 10x + 3$$

$$2x + 7$$

$$5x^2 - 8x + 10$$

Curve Fitting

- *Curve fitting* is the process of adjusting the coefficients in a mathematical function so that it lays as closely as possible to a set of data points.
- It is like creating a formula that tries to approximate the experimental results.
- You can then use that function as a mathematical model to represent the data.



Curve Fitting with Polynomials (The polyfit Command)

- There are two general ways to fit a polynomial to data points.
 - The polynomial must pass through every single data point.
 - The Polynomial does not need to pass through every data point.

The Fit Through All Data Points

- Given n data points (x_i, y_i) , you can make a polynomial of degree $n-1$ that will pass through all n points. Here are some examples:
 - Given two points, you can write a linear equation (polynomial of degree one) $y = mx + b$ that passes through both points.
 - Given three points, you can write a quadratic equation (polynomial of degree two) $y = ax^2 + bx + c$ that goes through all three points.
 - Given five points, a polynomial of degree 4 $y = ax^4 + bx^3 + cx^2 + dx + e$ would actually go through all 5 points.

The Fit Through Points Close Enough

- You can imagine with the previous method, that if you have 1000 data points, you would need a polynomial of degree 999. Not very practical! There is a better solution:
- Given a set of n data points (x_i, y_i) , you can often make a polynomial of degree less than $n-1$ that may not pass through any of the points but still approximates the data set overall. The most common method of doing this is called the *least-squares fit*.
- In this method, we do not merely change the values of the coefficients but also the formulas themselves (we try polynomials of different degrees to see what is the better fit).

Least-Squares Fit Method

- How to make a least-squares fit of a polynomial $p(x)$ of degree n to a set of data points (x_i, y_i) :
 - a. Compute the difference $p(x_i) - y_i$ at each data point. That difference is often called the *residual* or *error*.
 - b. Square each difference.
 - c. Sum the squares.
 - d. Find the values of the $n+1$ coefficients of $p(x)$ that minimize this sum.

EXAMPLE:

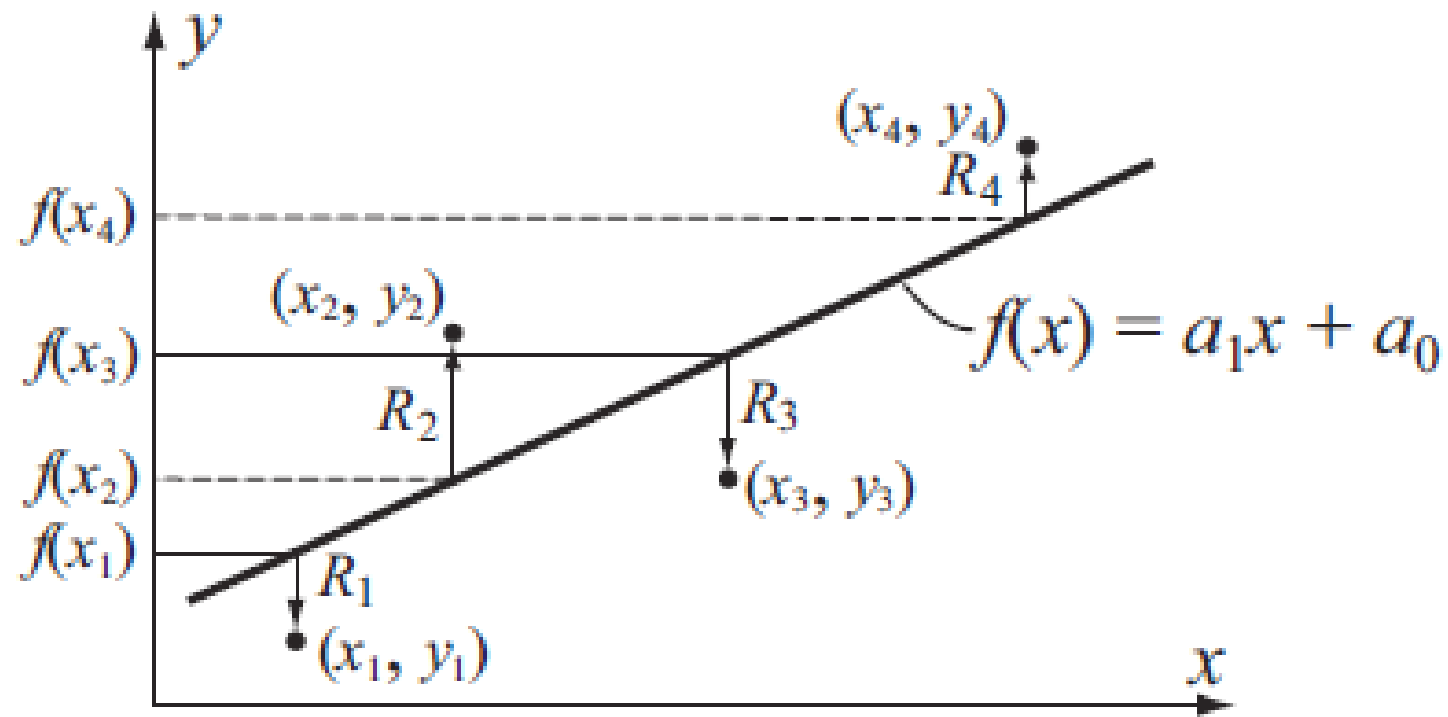


Figure 8-1: Least square fitting of first degree polynomial to four points.

The polyfit Command

- The MATLAB command `polyfit` computes the least-squares best fit of data points to a polynomial.

```
p = polyfit(x,y,n)
```

`p` is the vector of the coefficients of the polynomial that fits the data.

`x` is a vector with the horizontal coordinate of the data points (independent variable).
`y` is a vector with the vertical coordinate of the data points (dependent variable).
`n` is the degree of the polynomial.

The polyfit Command

- When using **polyfit** on a set of m data points, you can use any polynomial degree n such that $n \leq m-1$.

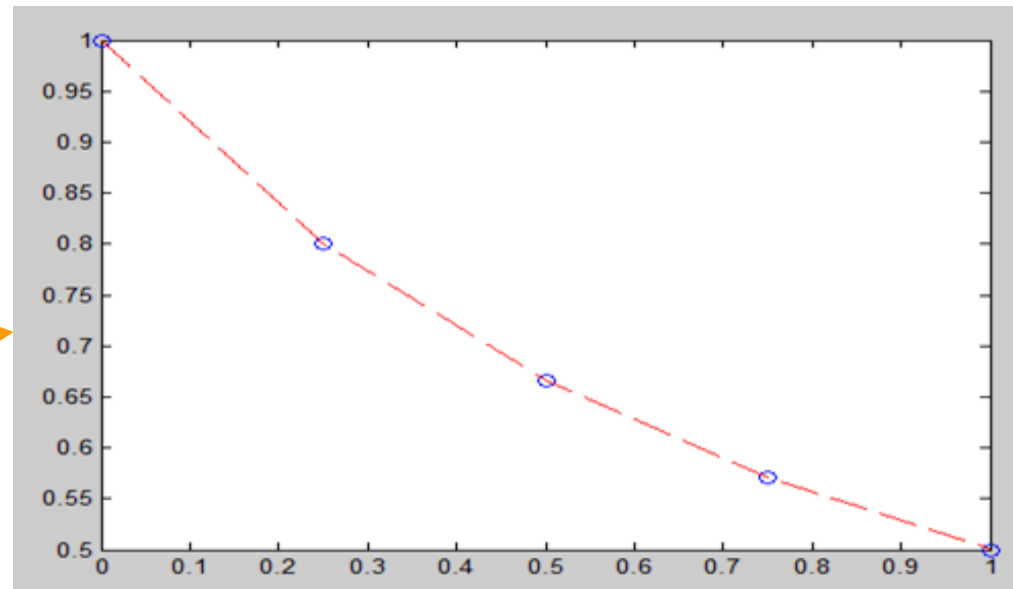
If $n = m-1$ the polynomial will go through all of the data points!

Note:

A polynomial of degree $m-1$, or similar high degree, may not necessarily provide the best overall fit because it may deviate substantially between data points.

The polyfit Command : Example

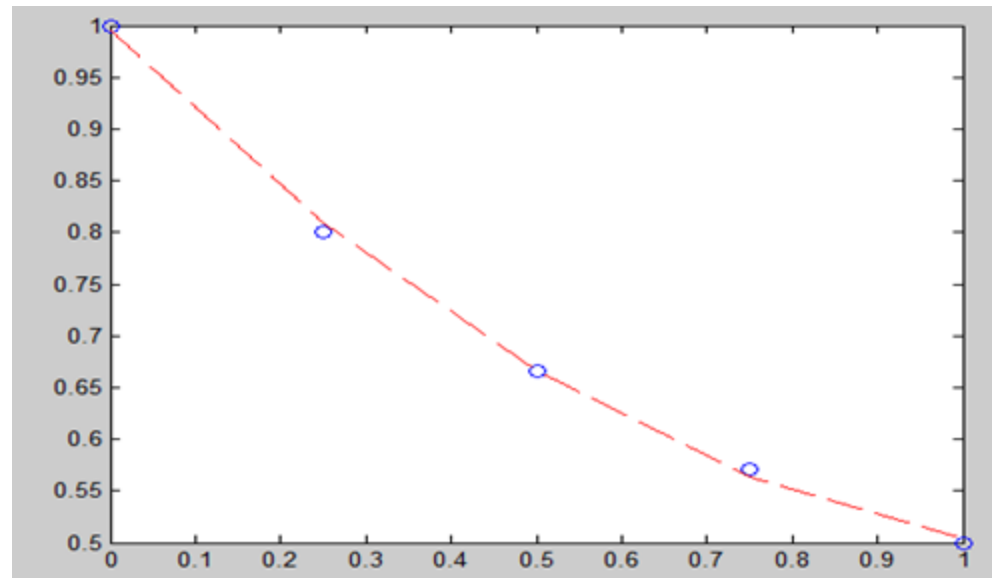
```
>> x = linspace(0,1,5)
x = 0      0.2500      0.5000      0.7500      1.0000
>> y = 1./(1+x)
y = 1.0000      0.8000      0.6667      0.5714      0.5000
>> p1 = polyfit(x,y,4)
p1 = 0.1524      -0.5333      0.8667      -0.9857      1.0000
>> plot(x,y,'o')
>> f1 = polyval(p1,x)
f1 = 1.0000      0.8000      0.6667      0.5714      0.5000
>> hold on
>> plot(x,f1,'r--')
```



The polyfit Command : Another Example

```
>> x = linspace(0,1,5)
x = 0      0.2500      0.5000      0.7500      1.0000
>> y = 1./(1+x)
y = 1.0000      0.8000      0.6667      0.5714      0.5000
>> p2 = polyfit(x,y,2)
p2 = 0.3374      -0.8288      0.9955
>> plot(x,y,'o')
>> f2 = polyval(p2,x)
f2 = 0.9955      0.8094      0.6654      0.5637      0.5041
>> hold on
>> plot(x,f2,'r--')
```

Almost
perfect
fit!

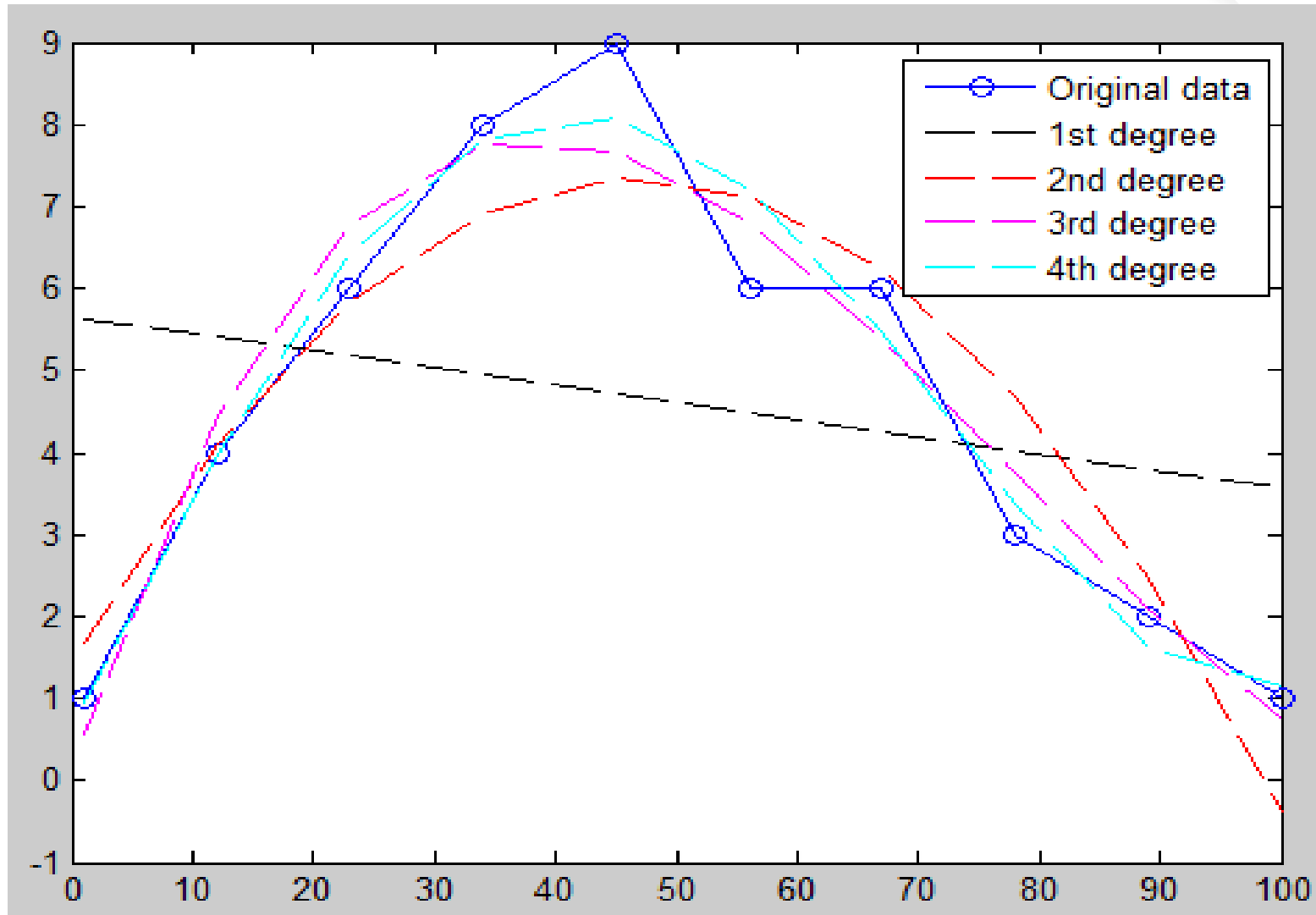


polyfit : Another Example

- The following script compares actual data points with different polynomial equations of degrees 1 to 4 (plot is on the next slide).

```
data_x = linspace (1,100,10);  
data_y = [1 4 6 8 9 6 6 3 2 1];  
  
plot(data_x, data_y, '-o');  
hold on  
p = polyfit(data_x, data_y, 1);  
plot(data_x, polyval(p,data_x), '--k');  
p = polyfit(data_x, data_y, 2);  
plot(data_x, polyval(p,data_x), '--r');  
p = polyfit(data_x, data_y, 3);  
plot(data_x, polyval(p,data_x), '--m');  
p = polyfit(data_x, data_y, 4);  
plot(data_x, polyval(p,data_x), '--c');  
legend ('Original data', '1st degree', '2nd degree', '3rd  
degree', '4th degree')  
hold off
```

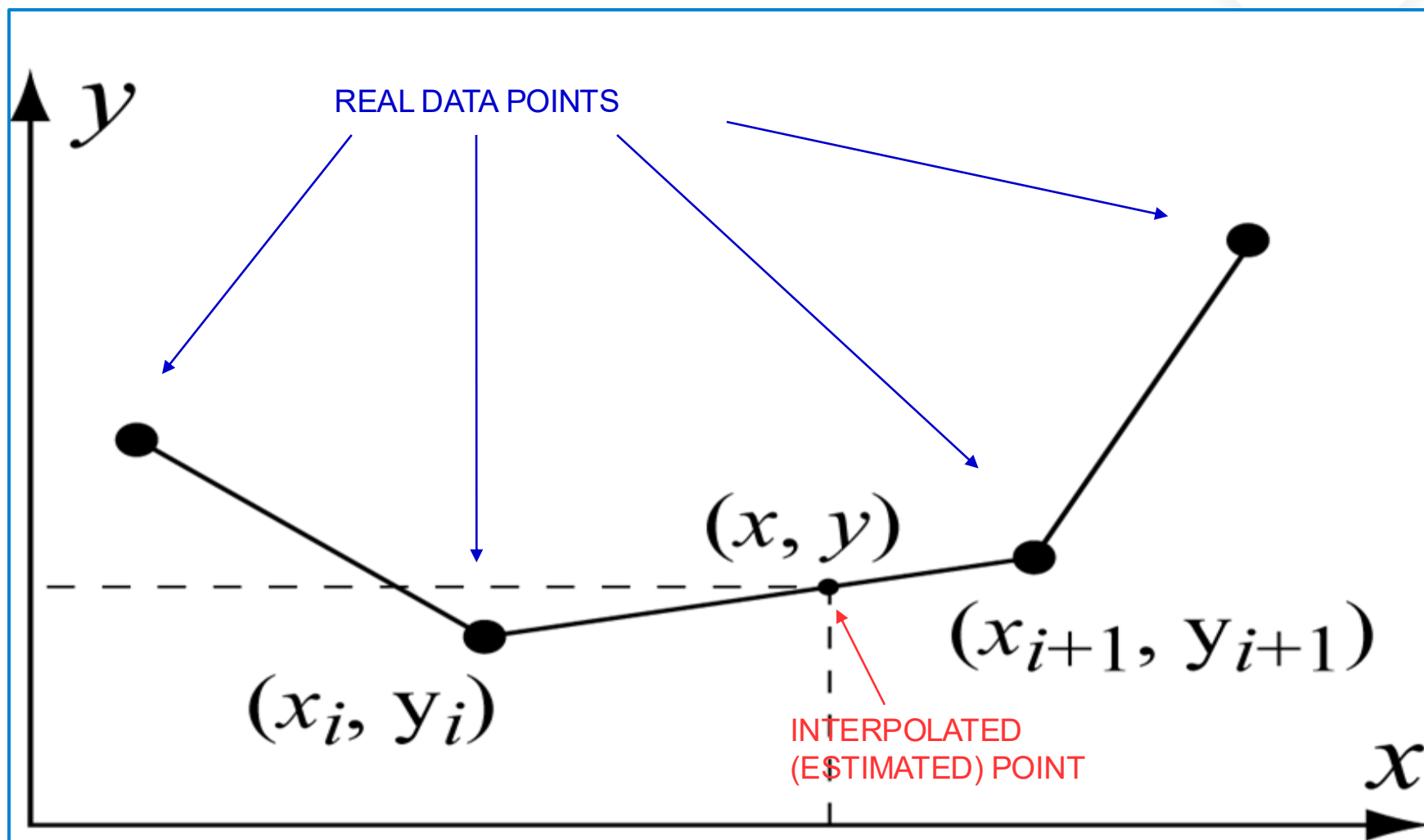
polyfit : Another Example



Interpolation

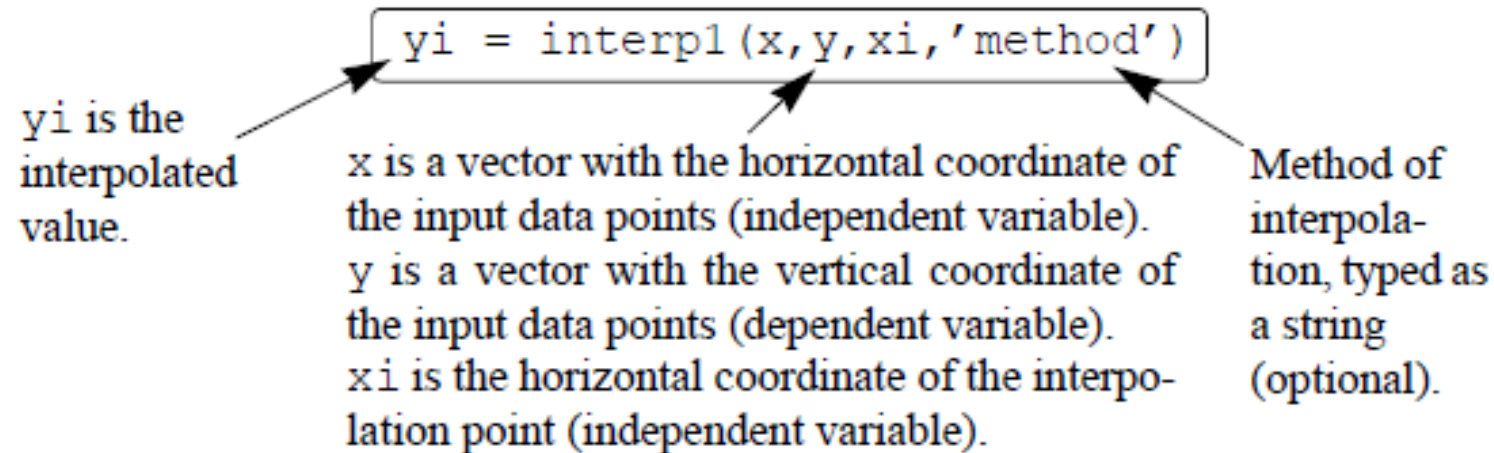
- Interpolation is the estimation of values between data points. MATLAB interpolation commands are based on polynomials.
- In one-dimensional interpolation each point has one independent variable (x) and one dependent variable (y).
- The simplest method of interpolation is the **linear interpolation**. In that case a straight line is assumed between each data point.

Linear Interpolation



Interpolation

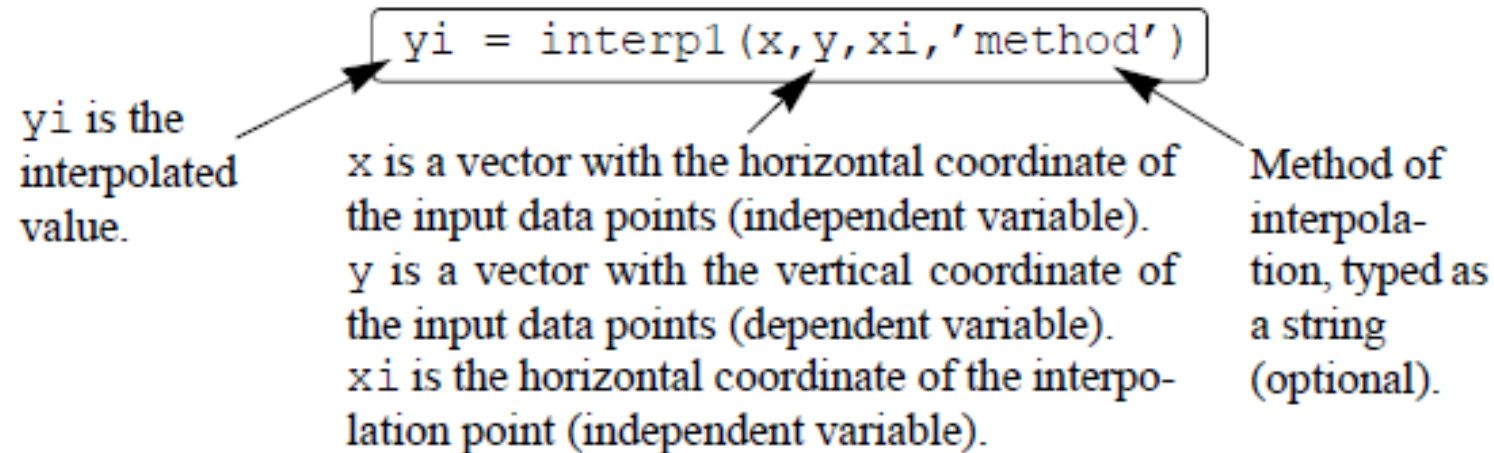
- The MATLAB function `interp1()` does the one-dimensional interpolation.



- The vector `x` must be monotonic (the elements in ascending or descending order).
- `xi` can be a scalar (interpolation of one point) or a vector (interpolation of many points). Respectively, `yi` is a scalar or a vector with the corresponding interpolated values.

Interpolation

- The MATLAB function `interp1()` does the one-dimensional interpolation.



- The vector `x` must be monotonic (the elements in ascending or descending order).
- `xi` can be a scalar (interpolation of one point) or a vector (interpolation of many points). Respectively, `yi` is a scalar or a vector with the corresponding interpolated values.

Interpolation Methods

- **linear**: Linear interpolation. Default method if not specified.
- **nearest**: Nearest neighbour interpolation. Matches the closest point.
- **next**: Next neighbour interpolation. Matches the next point.
- **previous**: Previous neighbour interpolation. Matches the previous point.
- **spline**: Uses a polynomial interpolation of lesser degree (like we did in curve fitting).
- **cubic**: Like a spline interpolation but with a polynomial of third degree only.

About our lecture

Polynomials, Curve
Fitting, Interpolation
and Numerical
Analysis



The Basic Fitting Interface

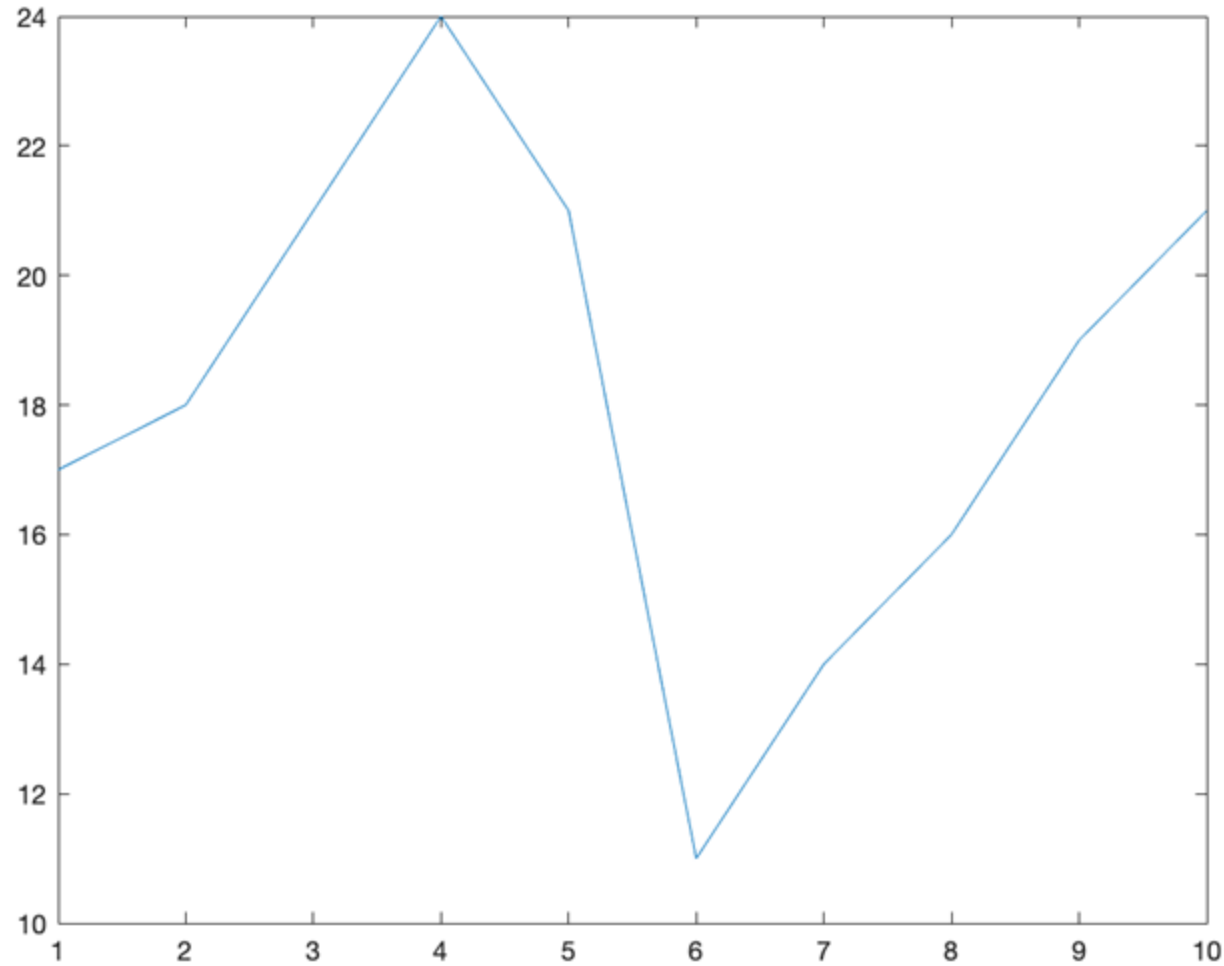
- MATLAB is a tool that can be used to perform curve fitting and interpolation interactively.
 - Do the curve fitting with polynomials of degrees up to 10.
 - Plot the various fits on the same graph so that they can be compared.
 - Plot and compare residuals of the various polynomial fits.
 - Calculate interpolated values with the various fits.
 - Add the equations of the polynomials to the plot.

Basic Fitting Interface Example (1)

- Let's do an example of working with the basic fitting interface step by step.
- Let's start with two vectors and a plot (the basic fitting interface uses the figure window as a starting point).

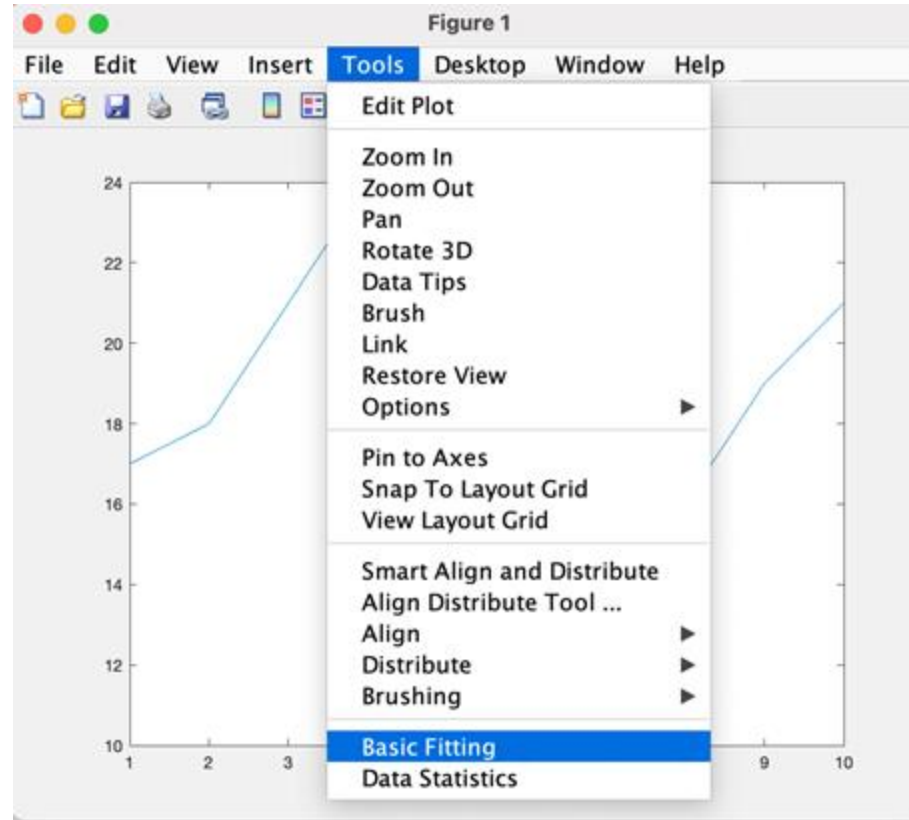
```
days = [1:10];  
temps = [17 18 21 24 21 11 14 16 19 21];  
plot (days, temps)
```


Basic Fitting Interface Example (2)



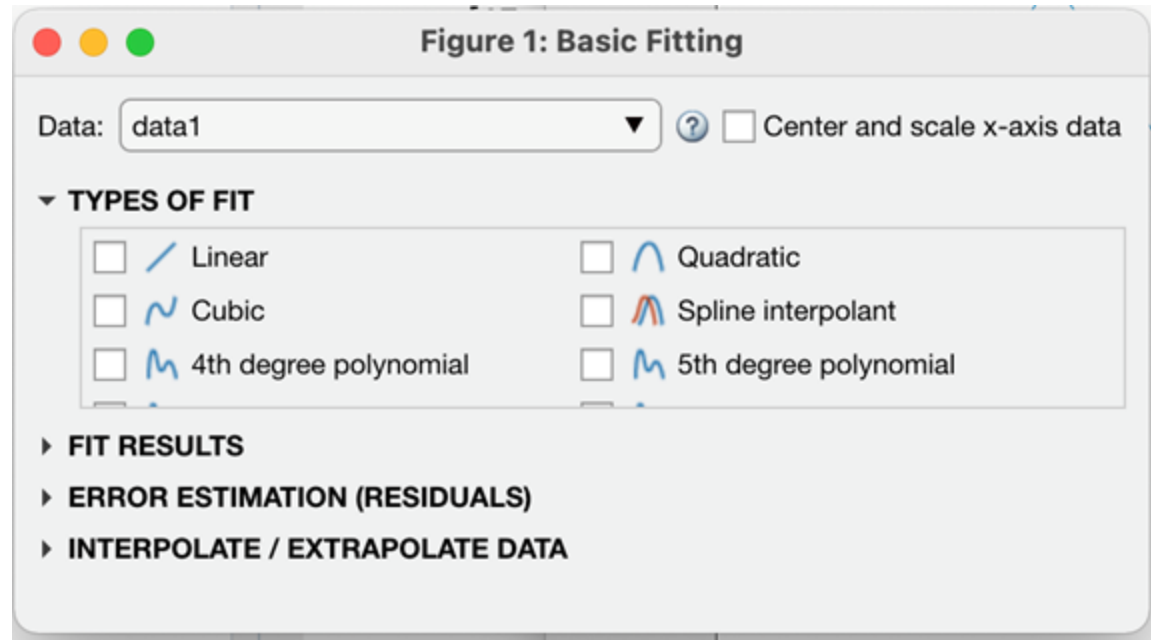
Basic Fitting Interface Example (3)

- Now for the basic fitting interface. From the figure window, choose the Basic Fitting option under the Tools menu. The Basic fitting dialog appears.

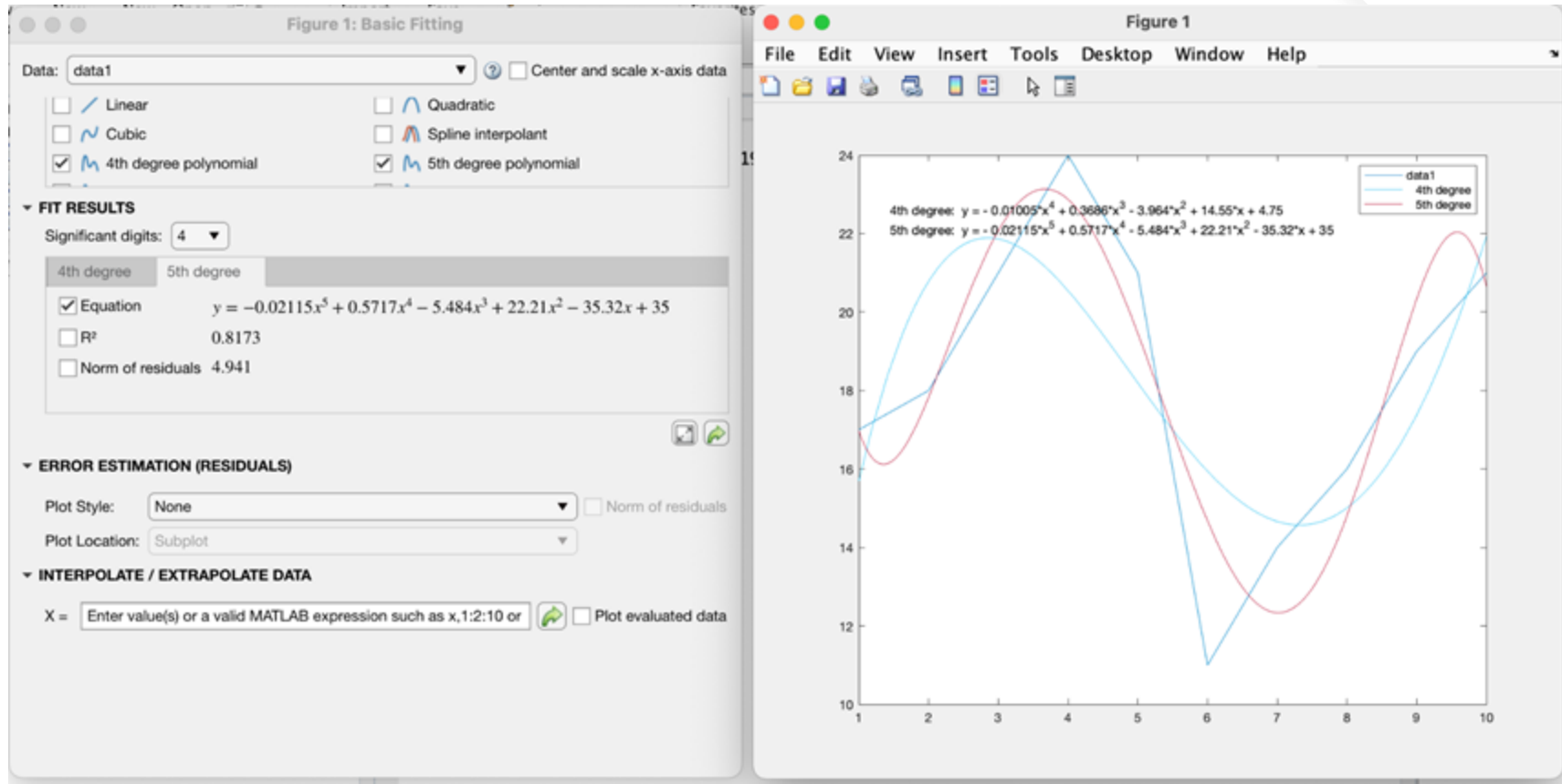


Basic Fitting Interface Example (4)

- Now select the type(s) of fit you want and see the equation and plot appear in the figure window. It is that simple. You can then pick the best choice (usually the lowest degree possible that is an acceptable fit).



Basic Fitting Interface Example (5)



- You can see the equation in the basic fitting dialog and the plot itself. The plot lines and the legend are updated live.

Basic Fitting Interface Options

- **Data:** If more than one data set is plotted, it lets you select which one to work on. You can only work on one data set at a time, but you can perform multiple fits simultaneously on that data.
- **Center and scale x-axis data:** Centers data at 0 mean and transforms it so that standard deviation is 1.

Basic Fitting Interface Options

- **Significant digits:** You can select the decimal precision of the coefficients.
- **Plot residuals:** Indicate whether or not to plot fit residuals and specify the style of the residual plot.
- **Show norm of residuals:** Indicate whether or not to display the norm of residuals. The norm is a measure of the quality of the fit. A smaller norm means a better fit.

More Basic Fitting Interface Options

- **Fit:** Select which fit to examine the details of.
- **Coefficients and norm of residuals:** Show numerical values of coefficients in the fit equation and value of the norm.
- **Find $y = f(x)$:** Provide ability to examine fit values of manually-entered independent-variable values.
- *The next slide shows an example of a Figure Window modified by changes in the Basic Fitting Interface.*

Basic Fitting Dialog with Options Selected

Figure 1: Basic Fitting

Data:  ☐ Center and scale x-axis data

☐ Linear ☐ Quadratic
☒ Cubic ☐ Spline interpolant
☒ 4th degree polynomial ☒ 5th degree polynomial

▼ FIT RESULTS

Significant digits:

	4th degree	5th degree	Cubic
☒ Equation	$y = -0.0101x^4 + 0.369x^3 - 3.96x^2 + 14.6x + 4.75$		
☐ R ²	0.556		
☐ Norm of residuals	7.7		

▼ ERROR ESTIMATION (RESIDUALS)

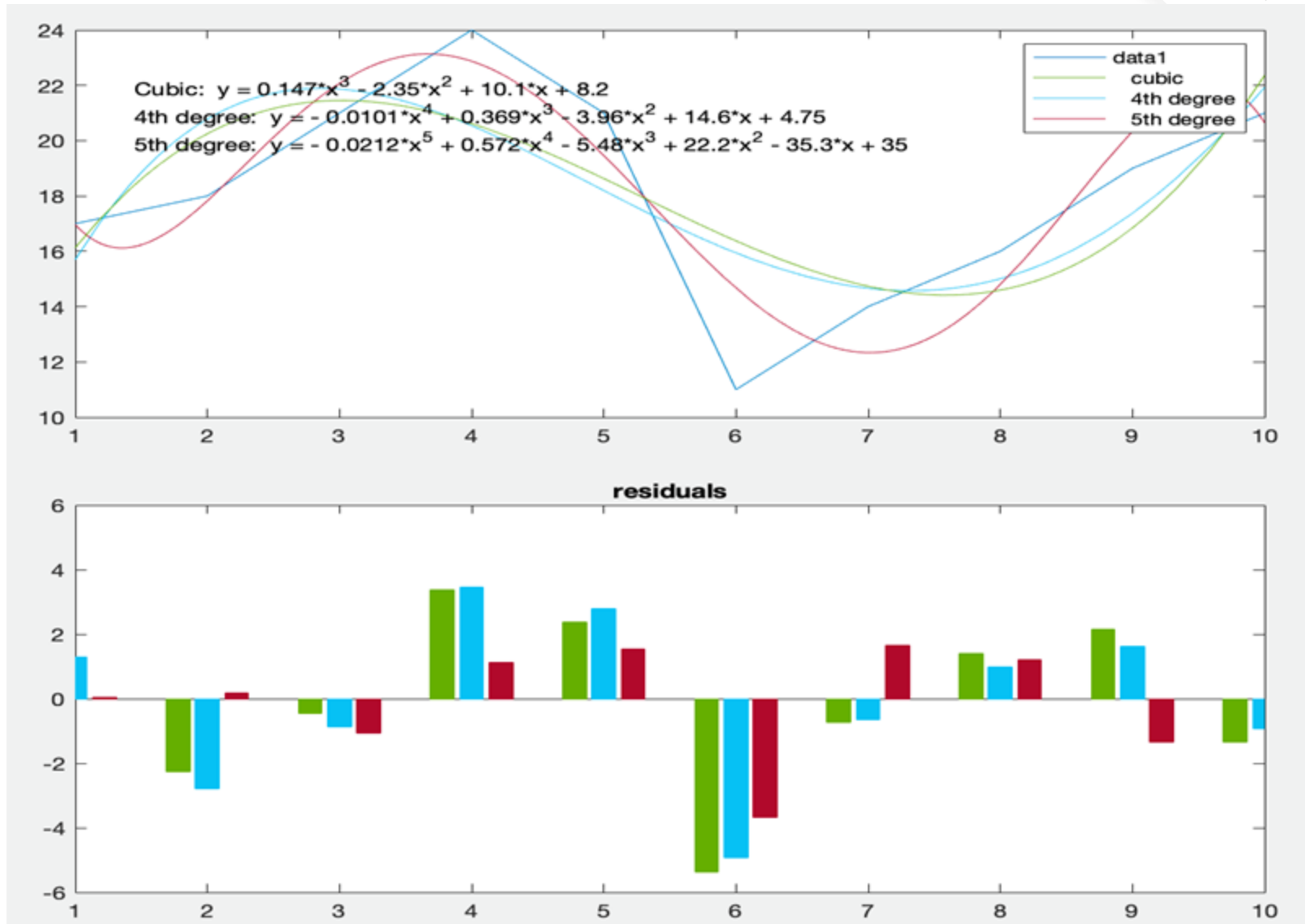
Plot Style: ☐ Norm of residuals

Plot Location:

▼ INTERPOLATE / EXTRAPOLATE DATA

X =  ☐ Plot evaluated data

The Updated Figure Window



Introduction to Numerical Analysis

- A numerical method is a technique for computing a numerical approximation of the solution to a mathematical problem.
- An equation can be written with one (independent) variable in the form $f(x) = 0$.
- A solution or root of the equation is a value of the variable x that makes the equation be true: $f(x) = 0$.
- Making the equation be true is called satisfying the equation.
- Graphically, a solution is a point where the function $f(x)$ touches or crosses the x -axis.

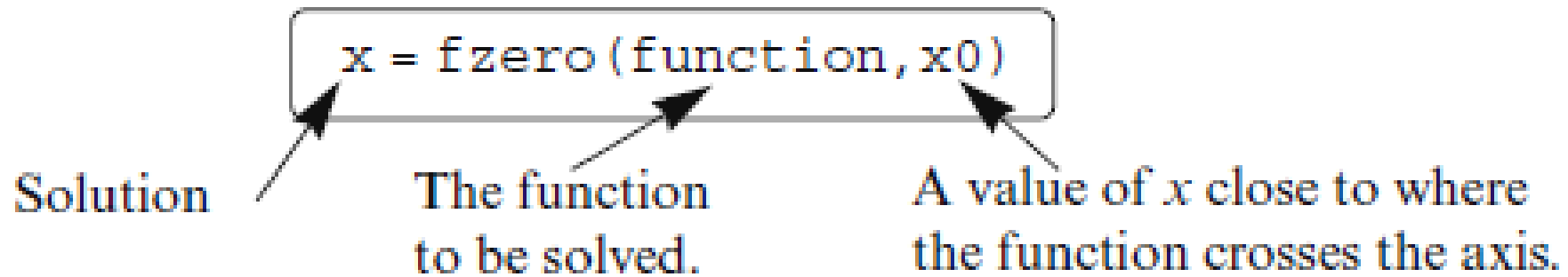
Numerical Analysis: Solutions

- The equation $f(x) = 0$ can have the possible number of solutions.
 - No solutions. $f(x)$ has 0 solutions if $f(x) = 5$.
 - One solution. $f(x) = 0$ has exactly 1 solution if $f(x) = x$.
 - Two solutions. $f(x) = 0$ has exactly 2 solutions if $f(x) = 1 - x^2$.
 - An infinite number of solutions.

$f(x) = 0$ has an infinite number of solutions if $f(x) = \sin(x)$.

Numerical Analysis: The fzero Command

- Use the MATLAB function `fzero()` to find a root of a function.



- `x`, the numerical solution, is a scalar.
- `function` is the function to be solved. The best way is to make an inline function and pass its name to `fzero`.

Numerical Analysis: The fzero Command

- A good way to get an initial value is to plot the function and see where it crosses the x-axis.
- fzero finds zeros of a function only if they cross the x-axis, not just touch it.
- If fzero can't find a solution, it returns NaN.
- `[x fval] = fzero (function,x0)` returns the value of function at x.

Numerical Analysis: The fzero Command

- Here are some simple uses of fzero:
 - Calculate π by finding the zero of the sine function near 3.

```
>> fun1 = @sin; % function
>> x0 = 3; % initial point
>> x = fzero(fun1,x0)
x = 3.1416
```
 - To find the zero of cosine between 1 and 2. Note that $\cos(1)$ and $\cos(2)$ differ in sign.

```
>> fun2 = @cos; % function
>> x0 = [1 2]; % initial interval
>> x = fzero(fun2,x0)
x = 3.1416
```

Numerical Analysis: The fzero Command

- Here is an example with a polynomial:
 $f(x) = x^3 - 2x - 5$.
- Find the zero of $f(x)$ near 2.

```
>> fun3 = @(x)x.^3 - 2*x - 5;
```

```
>> x0 = 2; % initial point
```

```
>> z = fzero(fun3,x0)
```

```
z = 2.0946
```

Numerical Analysis: Parabola Example

- Find a value of x that makes the parabola $1 - (x-1)^2 = 0$ have a value of 0.5.

```
>> parabola = @(x)1-(x-1)^2;
```

```
>> fun4 = @(x) parabola(x) - 0.5;
```

```
>> value = fzero(fun4,1)
```

```
value = 0.2929
```

```
>> y = 1 - 1 / sqrt(2)
```

```
y = 0.2929
```


Numerical Analysis: the fminbnd Command

- Often we want to find the minimum or maximum of a function $y = f(x)$ in a certain region of the horizontal axis.
- It can be done with
`[x fval] = fminbnd(function,x1,x2)` where **function** is a continuous function or a single independent variable, **x1** and **x2** the boundaries on the x-axis (x1 must be smaller than x2), **x** is the zero (solution) and **fval** the function value at the zero.

Numerical Analysis: The fzero Command

- Here is an example with a polynomial:
 $f(x) = x^3 - 2x - 5$.
- Find the zero of $f(x)$ near 2.

```
>> fun3 = @(x)x.^3 - 2*x - 5;
```

```
>> x0 = 2; % initial point
```

```
>> z = fzero(fun,x0)
```

```
z = 2.0946
```

Example of the fminbnd Command

```
>> f=@(x)x^3 - 12*x^2 + 40.25*x -  
36.5;
```

```
>> [x fval] = fminbnd( f, 3, 8 )
```

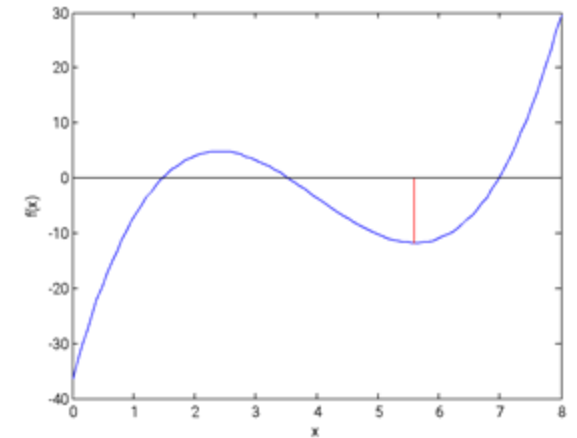
```
x = 5.6073
```

```
fval = -11.8043
```

```
>> [x fval] = fminbnd( f, 0, 8 )
```

```
x = 5.6073
```

```
fval = -11.8043
```



Numerical Analysis: Integration ($\int$)

- MATLAB can be used to perform integrals.
- The integral command in its simplest form `integral(func,xmin,xmax)` numerically integrates function `func` from `xmin` to `xmax` using global adaptive quadrature and default error tolerances.
- The commands `integral2` ($\iint$) and `integral3` ($\iiint$) also exist.

Numerical Analysis: Integration Example

- Let's have the function

```
>> f1 = @(x) exp(-x.^2).*log(x).^2;
```

- Evaluate the integral from 0 to infinity (Inf).

```
>> q1 = integral(f1,0,Inf)
```

```
q1 = 1.9475
```

$$f(x) = e^{-x^2}(\ln x)^2.$$

Numerical Analysis: Integration Example

- Let's have a function with one parameter c .

```
>> f2 = @(x,c) 1./(x.^3-2*x-c);
```

- Evaluate the integral from $x=0$ to $x=2$ at $c=5$.

```
>> q2 = integral(@(x) f2(x,5),0,2)
```

```
q2 = -0.4605
```

$$f(x) = 1/(x^3 - 2x - c)$$

Numerical Analysis: Integration Example

- Create the function $f(x) = \ln(x)$.

```
>> f3 = @(x)log(x);
```

- Evaluate the integral from $x=0$ to $x=1$ with the default error tolerances.

```
>> format long
```

```
>> q3 = integral(f3,0,1)
```

```
q3 = -1.0000000010959678
```

- More advanced commands and procedures will be covered in your future math classes.



End of lesson



Modular Programming and Functions



What Are Functions?

- A function is like a command that you create yourself. It can behave like a command already defined in MATLAB.
- There are also functions that programmers of the MATLAB organization have created and have placed in MATLAB for us to use. We've seen a number of those functions up to this point (`sqrt`, `sin`, ...), but in this chapter we will be mainly concerned with functions we will write ourselves.

What Are Functions?

- Functions are like “black boxes” that accept some data and return results.
- Many commands that we already know have one piece of data going in as **input** and 1 piece of data coming out as **result** (like *sqrt*) but we have encountered some with two inputs going in (like *nthroot*).
- Functions defined by you can have the number of inputs you need (0, 1, 2, 3, and more...).



Function Arguments

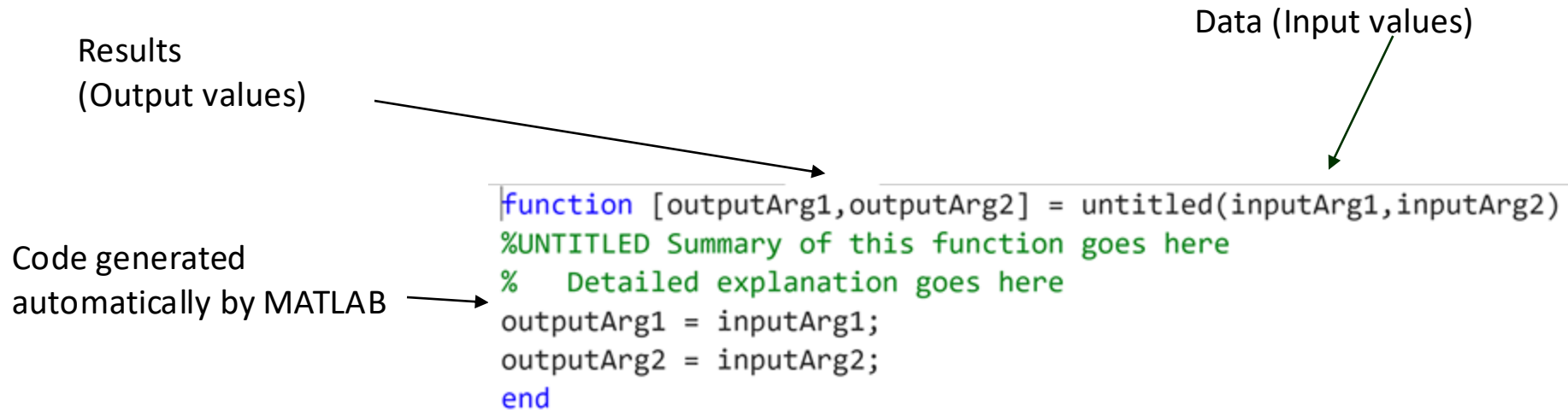
- A function is called or executed (run) by a program or another function. That program can pass its data to the function and receive the function's result.
- We call the values sent to the function when it is called, **arguments**. There can be one or more arguments.
- With `sqrt(x)`, `x` is the argument. The result is `sqrt(x)`, which can be assigned to a variable like in `y = sqrt(x);`

Why Use Functions?

- With a function, you can use the same code in more than one place in a program without rewriting the code. For example you know can use *sqrt* many times in the same program.
- You can also reuse the code by calling in the same function from different programs. For example, you can use *sqrt* in all your *m* and *mlx* files if needed.

Creating a Function

- A function is written almost exactly like a script, except that it starts with the **function** keyword. Like a script file, it is saved with a .m extension (it is recommended to use the same file name as your function's name). You can make the file in any text editor but it is easiest to do it with the MATLAB Editor/Debugger Window. To create a new function, select the *Function* item from the *New* menu.



A Simple Function

- This is a simple function that converts miles to kilometers.

RESULT

DATA (INPUT)

THE FUNCTION DEFINITION LINE

```
1 function [km] = miles_to_km (miles)
2 %miles_to_km converts distances in miles to distances in kilometers.
3 %Input:
4 %miles = distance in miles.
5 %Results:
6 %km = distance in kilometers.
7 km = miles * 1.60934;
8 end
```

H1 LINE

HELP TEXT

FUNCTION BODY AND ASSIGNMENT OF VALUE TO RESULT VARIABLE

We will explain all the elements in detail in the coming slides.

Sending Data to Functions

- Data input parameters (in the function header) and arguments (used when calling/invoking the function) are used to transfer data into the function from the calling program.
- You can have zero, one or more items of data sent to the function. If there are more than one, you separate the multiple data items with commas inside the parentheses of the function definition line.
- Data can be scalars, vectors, or arrays. Their values are specified by the calling program.

Getting Results from Functions

- Results (or output) are used to transfer data from the function to the calling program.
- You can have zero, one or more results from a function. If there are none, you can omit the assignment operator (=). If there is only one, you can omit the brackets. If more than one, you separate them with commas inside the brackets [] of the function definition line.
- Results can be scalars, vectors, or arrays.

Function Definition Line

```
function [output arguments] = function_name(input arguments)
```

The word `function` must be the first word, and must be typed in lower-case letters.

A list of output arguments typed inside brackets.

The name of the function.

A list of input arguments typed inside parentheses.
(also known as **parameters**)

- A function name is made up only of letters, digits, and underscores. It cannot have spaces. It follows the same rules as variable names.
- *Avoid making function names that are same as names of built-in functions.*

Function Definition Line

- The function definition line must be first executable line (not a blank line nor a comment line) of the function file. If not, MATLAB considers file to be a script file.
- The function definition line establishes the file as a function file, defines the name of the function, and defines the number and order of input items and results.
- As you see, a function file is really a script file but with special keywords and a special purpose.

○ The H1 Line and Help Text Lines

- *Help text lines* are comment lines (ones that start with a percent sign %) placed just below the function definition line. The first help text line, often called the H1 line, typically includes the program name and a brief description.
- End your help text with a blank line (without a %). The help system ignores any comment lines that appear after the help text block. Note that help text lines are optional.
- We will see an example of a function with H1 and help text lines on the next slide.

The Simplest Function: No Arguments, No Result

```
function why  
%displays 25 '?' on the screen.  
%Input: none.  
%Results: none.
```

```
disp (' ?????????????????????????????????????? ');  
end
```

You call (use) the function by simply entering the name of the function in the command window or your script.

```
>> why  
?????????????????????????????????????
```

H1 line and
help text lines.

If you type **help**
why, you will get
them in the
command
window.

A Simple Function: One Argument, No Result

- The *why* function only displays the same number of question marks every time.
- What if I wanted to display a different number of question marks that could be linked to a numerical value that I send to the function?
- All I would need to add is a parameter to the function definition line to specify the number of question marks. Now the function will have to be called with a corresponding argument to be passed (copied) to the parameter.
- See *why2* on the next slide. *why2*, like *why*, has no result.

A Simple Function: One Argument, No Result

```
function why2 (n)
%displays question marks on the screen.
%Input: n = number of question marks.
%Results: none.
```

```
for k=1:1:n
    fprintf ('?');
end
fprintf ('\n');
end
```

```
>> why2 (6)
??????
>> why2 (10)
??????????
```

This concludes
our overview of
MATLAB and a
taste of things to
come!

